CSF2753: Web Service Development
Coursework 1

Filippos Pantekis

Due: 28/10/2026 at 14:00pm

Overview

So far we have looked at web services in mostly abstract terms. We've talked a lot about distributed programming, how it has evolved over the years and what makes web services a good solution to many historical challenges. More recently we introduced the SOAP protocol and REST architecture as a means of implementing web services. While we have not yet started to write our own web services, we are in a reasonable position to start designing them.

In this coursework, we will **design** a RESTful web service that provides access to, and operations on, a dataset of **your choice**. The design should consider the **structure** of the dataset, the **operations** that clients can perform, and how these operations are **exposed** as RESTful endpoints. Applying lessons from socket-based programming, multithreading in Java, and managing HTTP information, we will also incorporate error handling, thread management and client-server request-response information in our design. Each of the requirements for this assignment are detailed in next few pages of this brief.

The **final deliverable** for this assignment will be a design document describing the web service you have designed.

Considerations

When it comes to developing web services, or APIs in general, beyond things functioning "correctly" there are no real right or wrong ways of doing things. However, there are certainly better or worse ways of designing and implementing your software. This assignment aims to help us understand the distinction between the two by applying the concepts we've learned, ideally for an application and context that is interesting **for you**. This assignment is intended to permit a lot of creative freedom, and that can lead to a lot of uncertainty. So, I reiterate here:

Hints

- Start early.
- Ask for help if you want it.
- Ask questions when you need clarification.
- Read this brief in its entirety **before** you start.

Tasks continue on the next page 👉

Task 1: Dataset Selection and Description [3 marks]

Task 1.1: Choosing a Dataset

Select a dataset that you find interesting and that can support **meaningful operations**. For example, your dataset should have enough information to appropriately support server responses that might filter, sort, or provide statistics about collections of data.

Note, these are just examples of what functionality your web service could provide. They are not minimum requirements (although they are essentially "free" suggestions).

Beware the CRUD

Your web service will need to use simple *Create*, *Read*, *Update*, *Delete* operations. However, the operations you design in this coursework, and implement in your next assignment, will need to be justified - why is this a distributed application? Why would you have a server that only returns single items from a spreadsheet?

You may choose from public datasets or, if you find yourself in excess of spare time, your own custom dataset. Note, choosing or building your own dataset **will not** earn additional marks.

Public Datasets

If you're interested in publicly available datasets, here are some resources to get you started.

- `https://www.kaggle.com/datasets`
- `https://www.gov.uk/search/research-and-statistics`
- `https://archive.ics.uci.edu/datasets`

Dataset Formats

Provided you are comfortable working with arbitrary dataset formats, you can do so. Otherwise, you may need to restrict yourself to choosing datasets that are downloadable in **Comma Separated Values** (`csv`) format. This is by far the most common format for these types of datasets.

Note: any tabular data can be exported to `csv` format from most spreadsheet editors (Microsoft Excel, Google Sheets, etc).

Task 1.2: Introducing the Dataset

Once you have chosen your dataset, provide a brief introduction that addresses the following:

- What is the dataset describing?
- What are the variable \ feature names?
- What are the data types?
- How many entries \ samples \ rows \ columns are there?

Task 1.3: Data Preprocessing

Finally, you should **consider** whether your dataset requires any *preprocessing*. In other words, do you need to manipulate the dataset in any way before responding to client requests. If you're using a `csv` file, then the answer is already *yes*.

Explain if the dataset needs preprocessing (e.g., cleaning, restructuring) to make it suitable for client requests.

Considerations

Be aware of the following points when choosing your dataset:
- Does it make sense to use this data for client requests?
- Is the dataset in a reasonable format?
- Is the preprocessing / cleaning going to be a lot of work?

Task 2: Supported Operations and Endpoints [6 marks]

Task 2.1: Define Supported Operations

Identify and **describe** at least 7 operations your service will support (e.g., retrieve records, search/filter records). This should be done in natural language and provided with your solutions for **Task 2.2**.

For each operation, **specify** which appropriate HTTP method each operation maps to.

HTTP Methods

You must map at least *one* operation to *each* of the HTTP methods `GET`, `PUT`, `POST` and `DELETE`.

Task 2.2: Endpoints

In order support your chosen operations, you will need to define a set of endpoints for clients to use. **Design** and **specify** the URI structure (i.e endpoints) for each of your chosen operations, ensuring they follow RESTful conventions.

As a base, you should design your endpoint with the following URI root:

`http://localhost:{portNumber}/{student_number}/`

Meaning, student `123456`, hosting their web service over port `8080`, would use the following URI as a root for all of their chosen endpoints:

`http://localhost:8080/123456/`

Endpoint - Operation Mismatch

Notice, there is no specified number of endpoints for you to define. This depends on the operations you have chosen to support! For example, a resource can be updated, read or deleted using the same URI. Therefore, if you have 7 unique operations, you will have *at most*, 7 endpoints.

Task 3: Request and Response Representations [5 marks]

Task 3.1: Request Format

For *each* operation, **provide** examples of the request format. This should include how data is passed and what parameters are required (e.g., `JSON`, query parameters).

It may not always be appropriate to include a request body in `JSON`, i.e. the client may not need to send anything other than an HTTP request. That said, you should *always* **provide** the HTTP header you expect the client to send.

Task 3.2: Response Format

Describe the format of the data returned by the service for each operation.
Provide examples of a typical `JSON/XML` response from the service.

Considerations

When designing and demonstrating the example request-response representations, be sure to justify your choices formats, particularly when it comes to ease of use for the client.
Note: If you expect any representations to be quite large (say you are retrieving a list of complex data), you may truncate the data shown in your example.

Task 4: Error Handling and Status Codes [3 marks]

Task 4.1: Identify Common Errors

For *each* operation, **identify** potential errors or failure scenarios that may feasibly occur. Considering both client-side and server-side actions that could raise exceptions. This could include, invalid input, resource not found, server errors.

Note

This does not need to be an exhaustive list, or one that reflects environmental faults like hardware or network failures.

Task 4.2: Assign Appropriate Status Codes

Key to the processing of HTTP responses are status codes. We expect to make full use of these for each of our supported endpoints.

Considering how your server will respond to each client request, **identify** the HTTP *status codes* your service will return for different outcomes. For example:

- `200 OK`
- `404 Not Found`
- `400 Bad Request`
- `500 Internal Server Error`

Provide a brief description of *when* and *why* each status code is used.

Task 5: Concurrency and Multithreading [3 marks]

Processing client requests Finally, having designed your web service, consider how threads will need to be managed and where race conditions are likely to occur.

For each operation, **explain** whether the process can be multithreaded. If so, **identify** and **describe** how race conditions may occur and how they can be mitigated.

Note

You **do not** need to provide extraneous technical detail concerning when the main thread of execution will start new threads. High-level explanations and descriptions are sufficient, but should be focused on data integrity and methodologies for preventing data races.

Please turn over ☞

Submission Instructions

Submit a detailed design document that includes all of the tasks above. The document should be clearly organised and explain your reasoning for the choices made in the design.

Remember to...

- Make sure the web service design follows RESTful principles.
- Provide justification for your design choices where necessary.
- Use appropriate examples in each task to demonstrate how the service works.

This coursework is the first step in preparing for the implementation phase in the next assignment. Focus on clear, logical, and realistic designs.

- Your design document should be a `pdf` with your student number as the filename.
- The document should be submitted via canvas, under `Assignments` → Coursework 1
- Submissions should be made before the deadline on **28/10/2026 at 14:00pm**

Warning

Follow these instructions carefully. Marks may be deducted for submissions that do not conform to the above.

Generative AI Usage Notice

⛔ The use of generative AI is **forbidden** for the entirety of this assignment.
For further information consult the university's Academic Integrity pages and if you require clarification please get in touch.

Academic Integrity Disclaimer

By submitting coursework, electronically and/or hardcopy, you state that you fully understand and are complying with the University's policy on Academic Integrity and Academic Misconduct. The policy can be found at' `https://myuni.swansea.ac.uk/academic-life/academic-misconduct` .

If you are unsure about Academic Misconduct make sure you have watched the induction lecture recording carefully. Handing in your coursework after the deadline will result in a mark of zero.